

MANUAL TÉCNICO

Aplicación de Rony Alvarez - Proyecto 2- EDD-

Índice

Estructuras de datos.....	4
1. Lista doblemente enlazada.....	4
2. Lista circular doblemente enlazada.....	5
3. Lista circular enlazada.....	7
4. Lista enlazada simple.....	8
5. Árbol AVL.....	9
6. Árbol BST.....	10
7. Árbol B(orden 4).....	11
8. Matriz dispersa.....	12
Registro y Login de usuarios.....	12
Carga masiva inicial - función 1.....	13
Gestionar inventario de equipos BST.....	14
Gestionar inventario de suministros B.....	16
Carga masiva de usuario departamental.....	18
Panel control de personal.....	18
Consultar y comparar inventario proveedor/fabricante.....	20
Reportes gráficos - GraphViz.....	20
Funciones de usuario departamentales.....	22
Estructura de carpetas y consideraciones.....	24

Introducción

Este manual acerca del software del Med trak, está hecho para que los desarrolladores interesados en el programa sean capaces de entender la lógica detrás de su funcionamiento, los métodos y funciones usadas en lenguaje de programación Perl.

Objetivo

El objetivo de este manual es ayudar a los desarrolladores, tutores auxiliares, y público en general a comprender el desarrollo de este software, y el pensamiento del desarrollador que lo ha creado.

También está hecho con el objetivo de mostrar al público general que el estudiante de EDD ha logrado utilizar los conceptos que fueron planteados antes del práctica, tanto como respetar las reglas de las estructuras que se nos fueron permitidas para usarse.

Dirigidos

Este manual está orientado a todas aquellas personas que estén interesadas en el funcionamiento del software, el desarrollo y lógica detrás de él. Para que en un futuro, por si les es de utilidad puedan utilizar este programa como guía para sus actividades.

Especificación técnica

Requisitos de Hardware

- Computadora de escritorio o portátil.
- 4GB de memoria RAM.
- 256GB de disco duro o más.
- Procesador intel i3, AMD ryzen 3, o superior.
- Resolución gráfica mínima de 1024 x 768 píxeles.

Requisitos de Software

- S.O. Windows 10 o superior.
- Tener instalado Perl Strawberry.
- Tener instalado GraphViz.
- Lenguaje de Programación Perl.
- Tener instalado Mojolicious de Perl

Especificación técnica

Estructuras de datos

1. Lista doblemente enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista doblemente enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces entre el que les sigue y el que los precede.

Esta es la clase nodo.pl

```
package nodo;
use strict;
use warnings;

sub new{
    my ($class, $valor)=@_;
    my $self = {
        valor => $valor,
        siguiente => undef,
        anterior => undef,
    };
    return bless $self, $class;
}

# getters / setters
sub value { $_[0]->{valor} }
sub set_value { $_[0]->{valor} = $_[1] }
sub prev { $_[0]->{anterior} }
sub set_prev { $_[0]->{anterior} = $_[1] }
sub next { $_[0]->{siguiente} }
sub set_next { $_[0]->{siguiente} = $_[1] }
1;
```

Esta otra, ya es la clase listaDoblementeEnlazada.

```

package listaDoblementeEnlazada;
use strict;
use warnings;

#llamar al nodo
require 'nodo.pl';

sub new{
    my($class) = @_ ;
    my($self) = {
        head => undef,
        tail => undef,
        size => 0,
    };
    return bless $self, $class;
}

#Método para ver si está vacía
sub estaVacía{ $_[0]->{size} == 0}

#método para ver el tamaño total
sub tamaño{ $_[0]->{size}}

```

Esta estructura de datos, además, cuenta con clases para insertar y eliminar elementos por delante o por detrás de la lista, un método para iterar la lista y otro para encontrar un elemento dentro de la lista.

2. Lista circular doblemente enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista circular doblemente enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces entre el que les sigue y el que los precede, pero, como detalle extra, el nodo del final y el nodo del principio, están conectados entre sí.

Esta es la clase de [nodoCircularDoble.pl](#)

```

package nodoCircularDoble;
use strict;
use warnings;

sub new {
    my ($class, $valor) = @_;
    my $self = {
        valor      => $valor,
        siguiente => undef,
        anterior  => undef,
    };
    return bless $self, $class;
}

```

Y esta es la clase de [listaCircularDoble.pl](#)

```

package listaCircularDoble;
use strict;
use warnings;

require 'nodoCircularDoble.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size=>0,
    };
    return bless $self, $class;
}

```

Esta estructura de datos, además, cuenta con clases para insertar por detrás, dos métodos para iterar la lista por delante y por detrás y otro para encontrar la cola de la lista, para la mejor manipulación de datos en el futuro.

3. Lista circular enlazada

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista circular enlazada consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces con el nodo que les sigue pero, como detalle extra, el nodo del final y el nodo del principio, están conectados entre sí.

Esta es la clase de [nodoCircular.pl](#)

```
package nodoCircular;
use strict;
use warnings;

sub new{
    my($class, $valor) = @_;
    my($self) = {
        valor => $valor,
        siguiente => undef,
    };
    return bless $self, $class;
}
```

Esta es la clase de [listaCircular.pl](#)

```
package listaCircular;
use strict;
use warnings;
require 'nodoCircular.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size => 0,
    };
    return bless $self, $class;
}
```

Esta estructura de datos, además, cuenta con clases para insertar por detrás, un método para iterar la lista por delante, uno para encontrar el tamaño de la lista y otro para encontrar la cola de la lista, para la mejor manipulación de datos en el futuro.

4. Lista enlazada simple

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La lista enlazada simple consiste en un conjunto de elementos nodos que están conectados entre sí por medio de enlaces con el nodo que les sigue, es la estructura más simple de este proyecto.

Esta es la clase de [nodoSimple.pl](#)

```
package nodoSimple;
use strict;
use warnings;

sub new{
    my($class, $valor) = @_;
    my $self = {
        valor => $valor,
        siguiente => undef,
    };
    return bless $self, $class;
}
```

Esta es la clase de [listaSimple.pm](#)

```
package listaSimple;
use strict;
use warnings;

require 'nodoSimple.pl';

sub new{
    my($class) = @_;
    my $self = {
        head => undef,
        size => 0,
    };
    return bless $self, $class;
}
```

Esta estructura de datos, además, cuenta con clases para insertar por detrás y por delante, un método para iterar la lista por delante y uno para encontrar el tamaño de la lista para la mejor manipulación de datos en el futuro.

5. Árbol AVL

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. El árbol AVL es una estructura de datos especial basada en un árbol BST con la gran diferencia en que este árbol se auto balancea, para que el factor de desbalance no sea mayor de 1, es decir, que la diferencia de altura entre el subárbol derecho e izquierdo no sea mayor a 1. Esta estructura cuenta con métodos para balancear, rotar a la izquierda, rotar a la derecha, insertar, eliminar nodos, balancear después de eliminar un nodo y los 3 tipos de recorridos para recorrer árboles(in Order, Pre Order, Post Order).

Esta es la clase de [nodoAvl.pl](#)

```
package nodoAvl;
use strict;
use warnings;

sub new {
    my ($class, $clave, $valor) = @_;
    my $self = {
        clave    => $clave,
        valor    => $valor,
        izq      => undef,
        der      => undef,
        altura   => 1,
    };
    return bless $self, $class;
}

1;
```

Y esta es la clase del árbol AVL.

```

package arbolAvl;
use strict;
use warnings;

require 'nodoAvl.pl';

sub new {
    my ($class) = @_;
    return bless { raiz => undef }, $class;
}

sub _altura {
    my ($self, $nodo) = @_;
    return 0 unless defined $nodo;
    return $nodo->{altura};
}

```

6. Árbol BST

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. El árbol BST es una estructura de datos base, en la que se insertan nuevos elementos, pero nunca se verifica que el árbol esté balanceado. Esta estructura cuenta con métodos para insertar, eliminar nodos, y los 3 tipos de recorridos para recorrer árboles(in Order, Pre Order, Post Order).

Esta es la clase de [nodoBST.pl](#)

```

package nodoBST;
use strict;
use warnings;

#Este es parecido a los otros nodos, menos por la altura

sub new {
    my ($class, $clave, $valor)= @_;
    my $self = {
        clave    => $clave,
        valor    => $valor,
        izq => undef,
        der  => undef,
    };
    return bless $self, $class;
}

1;

```

Esta es la clase de [arbolBST.pm](#)

```
package arbolBST;
use strict;
use warnings;
require 'nodoBST.pl';
sub new{
    my($class)=@_;
    return bless {
        raiz => undef
    }, $class;
}
```

7. [Árbol B\(orden 4\)](#)

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. El árbol B es una estructura de datos base más compleja que las otras, en la que se insertan una serie de elementos llamados claves y a la vez estas claves tienen una serie de hijos conectados entre cada clave, haciendo que este árbol no sea binario y la ramificación del árbol se haga para arriba. Esta estructura cuenta con métodos para insertar, eliminar nodos, y solamente con un tipo de recorrido(In order).

Esta es la clase del [nodoB.pl](#)

```
package nodoB;
use strict;
use warnings;

sub new {
    my ($class) = @_;
    return bless {
        claves => [],    # cada elemento: { clave => '...', valor => $objeto }
        hijos  => [],
        es_hoja => 1,
    }, $class;
}

sub num_claves { return scalar @{$_[0]->{claves}}; }

1;
```

Esta es la clase de [arbolB.pm](#)

```
package arbolB;
use strict;
use warnings;
my $ORDEN = 4; # max 3 claves, max 4 hijos por nodo

require 'nodoB.pl';

sub new {
    my ($class) = @_;
    return bless { raiz => nodoB->new() }, $class;
}
```

8. Matriz dispersa

Esta estructura de datos fue creada 100% desde 0 en el lenguaje de programación Perl, se utilizó la Poo para crear la clase desde 0 y así tener total control de la misma. La matriz dispersa es una estructura que cuenta con varias columnas y filas donde se van insertando datos dependiendo de las propiedades de los mismos, dependiendo del proveedor y fabricante en este caso para poder insertar los datos en un lugar u otro.

Esta es la clase de [nodoMatriz.pl](#)

```
package nodoMatriz;
use strict;
use warnings;

sub new{
    my($class, %args)=@_;
    my $self = {
        fila => $args{fila},
        columna => $args{columna},
        valor => $args{valor},
        derecha => undef,
        abajo => undef,
    };
    return bless $self, $class;
}
```

Esta es la clase de [matrizDispersa.pm](#)

```
package matrizDispersa;
use strict;
use warnings;
require 'nodoMatriz.pl';

sub new{
    my($class)=@_;
    my $self = {
        filas => {},
        columnas => {},
    };
    return bless $self, $class;
}
```

9. Grafo no dirigido

Estructura de datos que modela las relaciones de colaboración activa entre el personal del hospital. Cada nodo representa a un profesional registrado en el sistema (identificado por su número de colegio) y cada arista simboliza un vínculo de colaboración bidireccional y confirmado. Se implementa internamente mediante lista de adyacencia para optimizar el consumo de memoria y acelerar los recorridos. Esta estructura permite al administrador visualizar la conectividad interdepartamental, identificar profesionales aislados y generar sugerencias inteligentes mediante un BFS de dos saltos (profesionales que no son colaboradores

directos pero comparten dos o más contactos en común). Los nodos se colorean dinámicamente según el departamento asignado, facilitando la generación de reportes visuales en Graphviz.

```
TDA > lzw.pm
1 package lzw;
2 use strict;
3 use warnings;
4
5
6 sub new {
7     my ($class) = @_;
8     return bless {}, $class;
9 }
10
11 # --- Comprimir un string a una lista de códigos ---
12 sub comprimir {
13     my ($self, $texto) = @_;
14     return [] unless defined $texto && length $texto;
15
16     # Diccionario inicial: todos los bytes 0-255
17     my %dict;
```

10. Tabla hash

Estructura de acceso directo diseñada para agrupar y consultar al personal médico según su tipo de usuario (TIPO-01 a TIPO-04). Consta de cuatro buckets independientes (uno por tipo) y resuelve colisiones mediante encadenamiento externo. A diferencia del Árbol AVL, que optimiza búsquedas por número de colegio, esta tabla permite consultas en tiempo constante $O(1)$ para filtros por rol (ej. "mostrar a todos los enfermeros" o "contar técnicos de laboratorio"). Se sincroniza automáticamente con las operaciones del AVL y genera reportes gráficos que muestran la ocupación, distribución y estado de cada bucket, cumpliendo con el requisito de visualización de estructuras de hashing.

```
TDA > tablaHash.pm
1 package tablaHash;
2 use strict;
3 use warnings;
4
5 # Tamaño de la tabla interna (número de slots por bucket de tipo)
6 use constant TABLA_SIZE => 13; # primo para distribución uniforme
7
8 sub new {
9     my ($class) = @_;
10    # tabla: { 'TIPO-01' => [ [slot0,...], [slot1,...], ... ], ... }
11    my %tabla;
12    for my $t (qw(TIPO-01 TIPO-02 TIPO-03 TIPO-04)) {
13        # Cada slot es undef o una lista por chaining
14        $tabla{$t} = [ map { [] } 0 .. TABLA_SIZE - 1 ];
15    }
16    return bless {
17        tabla => \%tabla,
18        conteo => { 'TIPO-01'=>0, 'TIPO-02'=>0, 'TIPO-03'=>0, 'TIPO-04'=>0 },
19        colisiones => { 'TIPO-01'=>0, 'TIPO-02'=>0, 'TIPO-03'=>0, 'TIPO-04'=>0 },
20    }, $class;
21 }
22
```

11. Archivo de compresión LZW

Técnica de compresión sin pérdida aplicada al módulo de mensajería interna para garantizar la trazabilidad de conversaciones clínicas sin saturar el almacenamiento en disco. Al cerrar sesión, el historial de chats del usuario se serializa en una cadena estructurada, se comprime mediante un diccionario dinámico LZW y se guarda en un archivo individual chats/<numero colegio>.lzw. Al iniciar sesión, el archivo se lee,

descomprime y reconstruye en memoria, restaurando todas las conversaciones previas. Este mecanismo reduce drásticamente el tamaño de los datos en disco, mantiene copias independientes del historial por usuario (garantizando disponibilidad incluso si un colaborador no ha iniciado sesión) y cumple con los estándares de eficiencia y persistencia exigidos por el sistema hospitalario.

```
TDA > %\lzw.pm
1  package lzw;
2  use strict;
3  use warnings;
4
5
6  sub new {
7      my ($class) = @_;
8      return bless {}, $class;
9  }
10
11 # --- Comprimir un string a una lista de códigos ---
12 sub comprimir {
13     my ($self, $texto) = @_;
14     return [] unless defined $texto && length $texto;
15
16     # Diccionario inicial: todos los bytes 0-255
17     my %dict;
```

Registro y Login de usuarios

En el su de tipo post de /login se consulta directamente en el árbol AVL si el usuario que se está buscando está dentro del árbol, con el método del árbol llamado 'buscar', al cuál se le envía la clave para poder encontrar el usuario, si no lo encuentra, el endpoint envía una respuesta negativa, pero, si lo encuentra, retorna todos los datos del usuario y los mete en una variable de tipo session para conservar esa data durante toda la sesión.

```
post '/login' => sub ($c) {
    my $data = decode_json($c->req->body);
    my $user = $data->{usuario} // '';
    my $pass = $data->{password} // '';
```

```
547     async function doLogin() {
548         //usuario
549         const u = document.getElementById('lu').value.trim();
550         //contraseña
551         const p = document.getElementById('lp').value;
552         const msg = document.getElementById('lmsg');
553         const btn = document.getElementById('btnLogin');
```

Carga masiva inicial - función 1

Al ser una interfaz gráfica web, se utilizó también javascript para manejar el frontend de la aplicación. Para manejar cuestiones del espacio de carga de archivos, pre visualización de la información y envíos al backend, se utilizaron las funciones de

- DragOver, DragLeave, on Drop

```
function dragOver(e) {
    e.preventDefault();
    document.getElementById('upload-zone').classList.add('drag');
}
```

```
function dragLeave(e) {
    document.getElementById('upload-zone').classList.remove('drag');
}
function onDrop(e) {
    e.preventDefault();
    document.getElementById('upload-zone').classList.remove('drag');
    const file = e.dataTransfer.files[0];
    if (file) procesarArchivo(file);
}
```

Esto para manejar la carga de archivos desde el espacio de carga que se encuentra en el HTML de la misma página llamada admin.html.

- ArchivoSeleccionado

```
function archivoSeleccionado(e)
```

Esta función envía el archivo que está en el espacio de carga a la función para procesar el archivo.

- cargarCSV

```
async function cargarCSV()
```

Esta función envía los datos del archivo Json al endpoint de perl , espera la respuesta del servidor y llama a la notificación que genera la función de toast.

- procesar el archivo

```
function procesarArchivo(file)
```

Esta función toma el archivo que ha sido cargado en el espacio, muestra el peso, nombre y demás información técnica del archivo. Después de eso, muestra una previsualización del contenido del archivo.

- limpiarUpload

```
function limpiarUpload()
```

Esta función solamente limpia el área de carga del archivo y las notificaciones generadas.

- toast

Esta función genera una pequeña alerta en la parte inferior derecha de la pantalla, recibe el mensaje como parámetro y también el estado de la alerta(Bueno o malo).

- cargarMedicamentos

```
async function cargarMedicamentos()
```

Esta función carga la información de los medicamentos registrados a una tabla en el código HTML, esto después de consultar los datos a un endpoint de perl.

→→→→→→→→Backend→

Este es el endpoint para consultar un medicamento en específico.

```

=====CONSULTA DE MEDICAMENTOS=====
get '/medicamentos' => sub ($c) {
  my @lista;
  $listaMedicamentos->iterar(sub {
    my $nodo = shift; my $med = $nodo->value;
    push @lista, { codigo=>$med->codigoMedicina, nombre=>$med->nombreComercial, activo=>$med->principioActivo, l
  });
  $c->render(json => { ok=>1, medicamentos=>\@lista });
};

```

Este es el endpoint que carga los datos del json en la memoria del programa (Estructuras de datos), además, va verificando que no existan datos duplicados. Y luego da como resultado un mensaje de éxito o fracaso dependiendo de cómo haya terminado la operación.

```

172 post '/carga-masiva' => sub ($c) {
173   my $upload = $c->req->upload('json');
174
175   unless ($upload && $upload->filename =~ /\.json$/i) {
176     return $c->render(json => { ok=>0, mensaje=>'El archivo debe ser .json' });
177   }
178

```

Gestionar inventario de equipos BST

→→→→→→→**Backend**→→→

Este es el endpoint en el que se llama a x o y recorrido del árbol BST, según sea la petición del usuario en el frontend.

```

380 # Este endpoint es para recorrer el árbol de diferentes formas
381 get '/equipos' => sub ($c) {
382   my $recorrido = $c->param('recorrido') // 'inorden';
383   my $lista;
384
385   if ($recorrido eq 'preorden') { $lista = $arbolBST->preorden; }
386   elsif ($recorrido eq 'postorden') { $lista = $arbolBST->postorden; }
387   else { $lista = $arbolBST->inorden; }
388

```

Este es el endpoint en el que se utiliza el método de búsqueda del árbol BST para así buscar un equipo en específico dentro del árbol, si se encuentra, se retorna la información necesaria al frontend y si no se encuentra, se envía un mensaje de error.

```

405 # Este endpoint es para buscar los equipos uno a uno
406 get '/equipos/:codigo' => sub ($c) {
407   my $codigo = uc(trim($c->param('codigo')));
408   my $eq = $arbolBST->buscar($codigo);
409
410   unless (defined $eq) {
411     return $c->render(json => { ok=>0, mensaje=>'Equipo $codigo no encontrado' });
412   }

```

En este endpoint se registra un equipo en específico dentro del árbol BST, usando el método de insertar que tiene el propio árbol. Además este endpoint verifica que se hayan enviado todos los datos necesarios para la creación del equipo en memoria, si no se mandan los datos suficientes, se envía un mensaje de error al frontend.


```

426 # Este endpoint es para registrar un equipo de manera individual
427 post '/equipos' => sub ($c) {
428     my $d = decode_json($c->req->body);
429     my $codigo = uc(trim($d->{codigo} // ''));
430     my $nom = trim($d->{nombre} // '');
431
432     return $c->render(json => { ok=>0, mensaje=>'Código y nombre son obligatorios' });
433     unless $codigo && $nom;
434 }

```

En este endpoint se utiliza el id de un equipo para así eliminarlo del árbol BST, estos datos deben de llegar desde el frontend, si el usuario no los envía, se marca error, y si el usuario envía correctamente todos los datos, se elimina exitosamente el elemento(si es que este existe dentro del árbol).

```

456 # Esto es para poder eliminar un equipo
457 del '/equipos/:codigo' => sub ($c) {
458     my $codigo = uc(trim($c->param('codigo')));
459
460     unless (defined $arbolBST->buscar($codigo)) {
461         return $c->render(json => { ok=>0, mensaje=>"Equipo $codigo no encontrado" });
462     }
463
464     $arbolBST->eliminar($codigo);
465     $c->render(json => { ok=>1, mensaje=>"$codigo eliminado exitosamente" });
466 }

```

→→→→→→→**Frontend**→→→

```
async function registrarEquipo()
```

Esta función recibe los datos del formulario HTML de la parte de equipos , verifica que toda la información vaya completa y si es así, envía la petición al backend para que el equipo sea registrado. Al recibir la respuesta del backend, actualiza la tabla en la que se listan los equipos que están registrados y muestra un mensaje de éxito o fracaso según haya sido el caso.

```
async function buscarEquipo()
```

Esta función recibe el id del equipo que se desea buscar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el elemento y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, coloca los datos del equipo dentro de una tabla.

```
async function eliminarEquipo()
```

Esta función recibe el id del equipo que se desea eliminar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el , se elimina(si este fuese el caso) y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, muestra un mensaje de éxito en la operación.

```
async function cargarEquipos(recorrido = 'inorden')
```

Esta función recibe el tipo de recorrido que se debe de aplicarle al árbol, luego, envía esa información al backend, se espera la respuesta del backend, y cuando se obtienen todos los datos del backend, estos se muestran en el orden seleccionado por el usuario.

Gestionar inventario de suministros B

→→→→→Backend→→

Este es el endpoint en el que se llama a x o y recorrido del árbol B, según sea la petición del usuario en el frontend.

```
470 # para retornar el recorrido Inorder
471 get '/suministros' => sub ($c) {
472   my $lista = $arbolB->inorden;
473
474   my @resultado = map {
475     my $sum = $_->(valor);
476     {
477       codigo    => $sum->codigoSuministro,
478       nombre    => $sum->nombreSuministro,
479       fabricante => $sum->fabricanteSuministro,
480       precio    => $sum->precioSuministro,
481       cantidad  => $sum->cantidadSuministro,
482       vence     => $sum->fechaVencimientoSuministro,
483       minimo    => $sum->nivelMinimoReorden,
484     }
485   } @$lista;
486
487   $c->render(json => { ok=>1, suministros=>\@resultado });
488 };
489
```

Este es el endpoint en el que se utiliza el método de búsqueda del árbol B para así buscar un suministro en específico dentro del árbol, si se encuentra, se retorna la información necesaria al frontend y si no se encuentra, se envía un mensaje de error.

```
490 # para buscar un código en específico
491 get '/suministros/:codigo' => sub ($c) {
492   my $codigo = uc(trim($c->param('codigo')));
493   my $sum = $arbolB->buscar($codigo);
494
495   unless (defined $sum) {
496     return $c->render(json => { ok=>0, mensaje=>"Suministro $codigo no encontrado" });
497   }
498
499   $c->render(json => {
500     ok      => 1,
501     codigo  => $sum->codigoSuministro,
502     nombre  => $sum->nombreSuministro,
503     fabricante => $sum->fabricanteSuministro,
504     precio  => $sum->precioSuministro,
505     cantidad => $sum->cantidadSuministro,
506     vence   => $sum->fechaVencimientoSuministro,
507     minimo  => $sum->nivelMinimoReorden,
508   });
509 };
510
```

En este endpoint se registra un suministro en específico dentro del árbol B, usando el método de insertar que tiene el propio árbol. Además este endpoint verifica que se hayan enviado todos los datos necesarios para la creación del equipo en memoria, si no se mandan los datos suficientes, se envía un mensaje de error al frontend.

```
511 # Para el registro individual
512 post '/suministros' => sub ($c) {
513   my $d = decode_json($c->req->body);
514   my $codigo = uc(trim($d->{codigo} // ''));
515   my $nom = trim($d->{nombre} // '');
516
517   return $c->render(json => { ok=>0, mensaje=>"Código y nombre son obligatorios" })
518     unless $codigo && $nom;
519
520   my $cod_sum = $codigo =~ /^SUM/i ? uc($codigo) : "SUM$codigo";
521
522   return $c->render(json => { ok=>0, mensaje=>"$cod_sum ya existe en el inventario" })
523     if defined $arbolB->buscar($cod_sum);
524
525   my $sum = suministro->new(
526     tipo            => 'SUMINISTRO',
527     codigoSuministro => $cod_sum,
528     nombreSuministro => $nom,
529     fabricanteSuministro => trim($d->{fabricante} // ''),
530     precioSuministro  => $d->{precio} // 0,
531     cantidadSuministro => $d->{cantidad} // 0,
532   );
533   $arbolB->insertar($sum);
534   $c->render(json => { ok=>1, mensaje=>"Suministro registrado exitosamente" });
535 };
536
```

En este endpoint se utiliza el id de un equipo para así eliminarlo del árbol B, estos datos deben de llegar desde el frontend, si el usuario no los envía, se marca error, y si el usuario envía correctamente todos los datos, se elimina exitosamente el elemento(si es que este existe dentro del árbol).

```
540 # Para eliminar un suministro del árbol
541 del '/suministros/:codigo' => sub ($c) {
542   my $codigo = uc(trim($c->param('codigo')));
543
544   unless (defined $arbolB->buscar($codigo)) {
545     return $c->render(json => { ok=>0, mensaje=>"Suministro $codigo no encontrado" });
546   }
547
548   $arbolB->eliminar($codigo);
549   $c->render(json => { ok=>1, mensaje=>"$codigo eliminado exitosamente" });
550 };
551
```

→→→→→**Frontend**→→

```
async function registrarSuministro()
```

Esta función recibe los datos del formulario HTML de la parte de equipos , verifica que toda la información vaya completa y si es así, envía la petición al backend para que el suministro sea registrado. Al recibir la respuesta del backend, actualiza la tabla en la que se listan los equipos que están registrados y muestra un mensaje de éxito o fracaso según haya sido el caso.

```
async function buscarSuministro()
```

Esta función recibe el id del equipo que se desea buscar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el elemento y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, coloca los datos del equipo dentro de una tabla.

```
async function eliminarSuministro()
```

Esta función recibe el id del equipo que se desea eliminar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el , se elimina(si este fuese el caso) y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, muestra un mensaje de éxito en la operación.

```
async function cargarSuministros()
```

Esta función ejecuta el recorrido inOrder del árbol B, que es el único recorrido válido para este árbol.

Carga masiva de usuario departamental

→→→→→**Backend**→→

Este es el endpoint que carga los datos del json en la memoria del programa (Estructuras de datos), además, va verificando que no existan datos duplicados. Y luego da como resultado un mensaje de éxito o fracaso dependiendo de cómo haya terminado la operación.

```

552 ##### PARA REGISTRAR USUARIOS DENTRO DEL ÁRBOL AVL#####
553 post '/carga-usuarios' => sub ($c) {
554   my $upload = $c->req->upload('json');
555
556   #siempre se comprueba que el usuario envíe un archivo json
557   unless ($upload && $upload->filename =~ /\.json$/i) {
558     return $c->render(json => { ok=>0, mensaje=>'El archivo debe ser .json' });
559   }
560
561   my $data;
562   eval { $data = decode_json($upload->slurp); };
563   if ($?) {
564     return $c->render(json => { ok=>0, mensaje=>'JSON malformado: ' . $@ });
565   }

```

→→→→→Frontend→→

```
function procesarArchivoUsr(file)
```

Esta función toma el archivo que ha sido cargado en el espacio, muestra el peso, nombre y demás información técnica del archivo. Después de eso, muestra una previsualización del contenido del archivo.

```
async function cargarUsuarios()
```

Esta función toma el json del espacio que ha sido cargado, lo envía al backend para que los datos sean cargados, espera la respuesta del backend, muestra el resultado mediante una alerta de toast y por último, llama a la función que carga todos los nuevos datos dentro de la tabla del personal.

Panel control de personal

→→→→→Backend→→

Este es el endpoint en el que se listan todos los usuario registrados en el árbol en el recorrido especificado mediante el parámetro que recibe el endpoint.

```

# tomar a los usuarios en el recorrido deseado
get '/usuarios' => sub ($c) {
  my $recorrido = $c->param('recorrido') // 'inorden';
  my $lista;

  if ($recorrido eq 'preorden') { $lista = $arbolAVL->preorden; }
  elsif ($recorrido eq 'postorden') { $lista = $arbolAVL->postorden; }
  else { $lista = $arbolAVL->inorden; }

  my @resultado = map { {
    numero_colegio => $_->{clave},
    nombre => $_->{valor}{nombre},
    tipo => $_->{valor}{tipo},
    departamento => $_->{valor}{depto},
    especialidad => $_->{valor}{espec} // '',
  }} @$lista;

  $c->render(json => { ok=>1, usuarios=>\@resultado });
};

```

Este es el endpoint en el que se utiliza el método de búsqueda del árbol AVL para así buscar un usuario en específico dentro del árbol, si se encuentra, se retorna la información necesaria al frontend y si no se encuentra, se envía un mensaje de error.

```

632 # buscar solamente a un usuario de entre todo el árbol
633 get '/usuarios/:colegio' => sub ($c) {
634   my $colegio = $c->param('colegio');
635   my $u       = $arbolAVL->buscar($colegio);
636
637   unless (defined $u) {
638     return $c->render(json => { ok=>0, mensaje=>"Usuario $colegio no encontrado" });
639   }
640
641   $c->render(json => {
642     ok           => 1,
643     numero_colegio => $colegio,
644     nombre       => $u->{nombre},
645     tipo         => $u->{tipo},
646     departamento => $u->{depto},
647     especialidad => $u->{espec} // '',
648   });
649 };
650

```

En este endpoint se utiliza el id de un equipo para así eliminarlo del árbol AVL, estos datos deben de llegar desde el frontend, si el usuario no los envía, se marca error, y si el usuario envía correctamente todos los datos, se elimina exitosamente el elemento(si es que este existe dentro del árbol).

```

671 # eliminar al usuario que está en el campo de buscar
672 del '/usuarios/:colegio' => sub ($c) {
673   my $colegio = $c->param('colegio');
674
675   unless (defined $arbolAVL->buscar($colegio)) {
676     return $c->render(json => { ok=>0, mensaje=>"Usuario $colegio no encontrado" });
677   }
678
679   $arbolAVL->eliminar($colegio);
680   $c->render(json => { ok=>1, mensaje=>"$colegio eliminado exitosamente" });
681 };
682

```

→→→→→**Frontend**→→

```

async function registrarUsuario()

```

Esta función recibe los datos del formulario HTML de la parte de equipos , verifica que toda la información vaya completa y si es así, envía la petición al backend para que el usuario sea registrado.

```

async function buscarUsuario()

```

Esta función recibe el id del usuario que se desea buscar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el elemento y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, coloca los datos del equipo dentro de una tabla.

```

async function eliminarUsuario()

```

Esta función recibe el id del usuario que se desea eliminar, si no recibe ese dato, marca un error. Pero, si se escribe un id en el campo, este se envía a perl, en el backend se busca el , se elimina(si este fuese el caso) y se envía una respuesta. Si no se encuentra el elemento, marca que no se encontró, pero, si lo encuentra, muestra un mensaje de éxito en la operación.

```

async function cargarPersonal(recorrido = 'inorden')

```

Esta función recibe el tipo de recorrido que se debe de aplicar al árbol, luego, envía esa información al backend, se espera la respuesta del backend, y cuando se obtienen todos los datos del backend, estos se muestran en el orden seleccionado por el usuario.

Consultar y comparar inventario proveedor/fabricante

→→→→→**Backend**→→→

Este es el endpoint en el que se accede al método de obtenerMatriz de la matriz dispersa, este método convierte la matriz dispersa en una estructura json que puede ser enviada al frontend de regreso.

```
683 #=====FUNCIÓN 7 PARA OBTENER LA MATRIZ DISPERZA=====
684 #=====
685 ✓ get '/matriz' => sub ($c) {
686   my $data = $matrizDispersaMed->obtenerMatriz();
687   $c->render(json => { ok=>1, %$data });
688 };
689
```

→→→→→**Frontend**→→→

```
async function cargarMatriz()
```

Esta función manda la petición al backend para que se procese la matriz en formato de json y sea devuelta al frontend, si la operación es exitosa, la información de la matriz es puesta en una tabla en el HTML del frontend.

Reportes gráficos - GraphViz

En esta parte, se muestran varias secciones, una para cada reporte asignado a esta fase del proyecto, para la parte de los árboles las funciones del frontend y del backend son en esencia lo mismo. → Los reportes de las estructuras de la fase 3 se gestionan hasta el final del archivo [app.pl](#).

Este es un ejemplo de una función del frontend de la generación de un reporte del árbol AVL.

```
async function generarReporteAVL() {
  mostrarMsg('rpt-avl-msg', 'ok', '> GENERANDO REPORTE...');
  document.getElementById('rpt-avl-wrap').style.display = 'none';

  try {
    const res = await fetch('/reporte/avl');
    const data = await res.json();

    if (data.ok) {
      const img = document.getElementById('rpt-avl-img');
      const dl = document.getElementById('rpt-avl-dl');
      img.src = data.imagen;
      dl.href = data.imagen;
      document.getElementById('rpt-avl-wrap').style.display =
'block';
      ocultarMsg('rpt-avl-msg');
    }
  }
}
```

```

        toast('> REPORTE AVL GENERADO', 'ok');
    } else {
        mostrarMsg('rpt-avl-msg', 'error', '> ERROR :: ' +
data.mensaje.toUpperCase());
    }
} catch(e) {
    mostrarMsg('rpt-avl-msg', 'error', '> ERROR :: SIN CONEXIÓN CON
EL SERVIDOR');
}
}

```

Y este es un ejemplo de una de esas funciones del backend.

```

715 #=====REPORTE DEL ARBOL AVL=====
716 #=====
717 get '/reporte/avl' => sub ($c) {
718     # Verificar que graphviz esté instalado
719
720     my $raiz = $arbolAVL->raiz;
721     unless (defined $raiz) {
722         return $c->render(json => { ok=>0, mensaje=>'El árbol AVL está vacío' });
723     }
724 }

```

Luego, de crear las imágenes en el backend, estas son enviadas dentro de la respuesta Json al frontend mediante un hash de base64, y esa misma imagen es cargada al HTML del frontend, además de darle una opción al usuario para que pueda descargar la misma.

Con la matriz la cosa cambia un poco, ya que la generación de su archivo .dot y su imagen es diferente, al contar con filas y columnas, estas deben de ser procesadas antes, para después ir procesando las relaciones entre cada nodo y así, unir toda la información. ->Luego, cargams

```

1025 #=====REPORTE DE LA MATRIZ DISPERSA =====
1026 get '/reporte/matriz' => sub ($c) {
1027     my $data = $matrizDispersaMed->obtenerMatriz();
1028     my @filas = @{$data->{filas}};
1029     my @columnas = @{$data->{columnas}};
1030     my @celdas = @{$data->{celdas}};
1031
1032     unless (@filas && @columnas) {
1033         return $c->render(json => { ok=>0, mensaje=>'La matriz está vacía' });
1034     }

```

Funciones de usuario departamentales

Al ser la mayoría de funciones de los usuarios departamentales, consultas de equipos, suministros, etc. Se utilizaron varios endpoints de otras funciones, y solamente se cargó la información que devuelven esos endpoints al frontend.

En el caso del frontend, en el JS, al iniciar sesión, siempre se comprueba el Tipo de usuario que se acaba de logear, para así mostrarle determinado contenido. Tanto en el nav como en las secciones con las que puede interactuar.

```

function aplicarPermisos(depto){
  // tenemos que ver qué contenido verá cada departamento
  const permisos = {
    'DEP-MED': { med: true, eq: false, sum: true },
    'DEP-CIR': { med: false, eq: true, sum: true },
    'DEP-LAB': { med: false, eq: true, sum: false },
    'DEP-FAR': { med: true, eq: false, sum: false },
    'DEP-ADM': { med: true, eq: true, sum: true },
  };

  const p = permisos[depto] || { med: false, eq: false, sum: false };
  // ya si no encuentra depto, no verá nada

  // Mostrar u ocultar secciones
  document.getElementById('sec-medicamentos').style.display = p.med ?
'block' : 'none';
  document.getElementById('sec-equipos').style.display = p.eq ?
'block' : 'none';
  document.getElementById('sec-suministros').style.display = p.sum ?
'block' : 'none';

  // Actualizar nav, para ocultar botones sin permiso
  const navMed =
document.querySelector('[onclick*="sec-medicamentos"]');
  const navEq = document.querySelector('[onclick*="sec-equipos"]');
  const navSum =
document.querySelector('[onclick*="sec-suministros"]');
  if (navMed) navMed.style.display = p.med ? 'block' : 'none';
  if (navEq) navEq.style.display = p.eq ? 'block' : 'none';
  if (navSum) navSum.style.display = p.sum ? 'block' : 'none';

  // Hacer scroll a la primera sección visible
  if (p.med) irA('sec-medicamentos', navMed);
  else if (p.eq) irA('sec-equipos', navEq);
  else if (p.sum) irA('sec-suministros', navSum);
  else irA('sec-perfil',
document.querySelector('[onclick*="sec-perfil"]'));
}

```

Y en la parte del backend, lo único nuevo que se agregó, fué el endpoint que edita la información del usuario.


```

#=====EDITAR EL PERFIL DEL USUARIO=====
# PUT /perfil - editar nombre y contraseña
put '/perfil' => sub ($c) {
  my $d = decode_json($c->req->body);
  my $colegio = trim($d->{colegio} // '');
  my $pass_actual = $d->{pass_actual} // '';
  my $nuevo_nom = trim($d->{nuevo_nombre} // '');
  my $nueva_pass = $d->{nueva_pass} // '';

  my $u = $arbolAVL->buscar($colegio);
  return $c->render(json => { ok=>0, mensaje=>'Usuario no encontrado' })
    unless defined $u;

  return $c->render(json => { ok=>0, mensaje=>'Contraseña actual incorrecta' })
    unless $u->{pass} eq $pass_actual;

  $u->{nombre} = $nuevo_nom if $nuevo_nom;
  $u->{pass} = $nueva_pass if $nueva_pass;

  $c->render(json => { ok=>1, mensaje=>'Perfil actualizado exitosamente' });
};

```

Carga masiva de relaciones entre usuarios

→→→→→**Backend**→→→

Este es el endpoint que carga los datos de las relaciones del json en la memoria del programa (Estructuras de datos), además, va verificando que no existan datos duplicados. Y luego da como resultado un mensaje de éxito o fracaso dependiendo de cómo haya terminado la operación.

```

1389 #
1390 # CARGA MASIVA DE RELACIONES (Grafo)
1391 #
1392 post '/carga-relaciones' => sub ($c) {
1393   my $upload = $c->req->upload('json');
1394   unless ($upload && $upload->filename =~ /\.json$/i) {
1395     return $c->render(json => { ok=>0, mensaje=>'El archivo debe ser .json' });
1396   }
1397
1398   my $data;
1399   eval { $data = decode_json($upload->slurp); };
1400   if ($@) {
1401     return $c->render(json => { ok=>0, mensaje=>'JSON malformado: ' . $@ });
1402   }
1403
1404   unless (ref($data) eq 'ARRAY') {
1405     return $c->render(json => { ok=>0, mensaje=>'El JSON debe ser un array de relaciones' });
1406   }
1407 }

```

A su vez, toda la información relacionada con el grafo no dirigido se encuentra en las siguientes líneas del proyecto, específicamente en el archivo [app.pl](#)

```

1443 # -----
1444 # GRAFO - Endpoints
1445 # -----
1446
1447 # Lista de todos los nodos del grafo
1448 get '/grafo/nodos' => sub ($c) {
1449     $c->render(json => { ok=>1, nodos => $grafo->todos_nodos() });
1450 };
1451
1452 # Colaboradores directos de un usuario
1453 get '/grafo/colaboradores/:colegio' => sub ($c) {
1454     my $col = $c->param('colegio');
1455     $c->render(json => { ok=>1, colaboradores => $grafo->colaboradores($col) });
1456 };
1457
1458 # Sugerencias BFS 2 saltos
1459 get '/grafo/sugerencias/:colegio' => sub ($c) {
1460     my $col = $c->param('colegio');
1461     $c->render(json => { ok=>1, sugerencias => $grafo->sugerencias($col) });
1462 };
1463
1464 # Lista de adyacencia completa
1465 get '/grafo/adyacencia' => sub ($c) {
1466     $c->render(json => { ok=>1, adyacencia => $grafo->lista_adyacencia() });
1467 };

```

Asignar departamento a un usuario

Este es el endpoint que se encarga de asignar un departamento a un usuario que no cuente con el mismo, esto desde la parte de administración.

```

1514 # Asignar departamento a un usuario (activa acceso)
1515 put '/usuarios/:colegio/asignar-depto' => sub ($c) {
1516     my $col = $c->param('colegio');
1517     my $d = decode_json($c->req->body);
1518     my $depto = trim($d->{departamento} // '');
1519
1520     return $c->render(json => { ok=>0, mensaje=>'Departamento inválido' })
1521         unless $depto =~ /^DEP-(ADM|MED|CIR|LAB|FAR)$/;
1522
1523     my $u = $arbolAVL->buscar($col);
1524     return $c->render(json => { ok=>0, mensaje=>"Usuario $col no encontrado" })
1525         unless defined $u;
1526
1527     $u->{depto} = $depto;
1528     $grafo->actualizar_depto($col, $depto);
1529
1530     $c->render(json => { ok=>1, mensaje=>"Departamento asignado: $depto a $col" });
1531 };

```

Asignar departamento a un usuario

Este es el endpoint que se encarga de buscar el bucket correcto dentro de la tabla hash, según sea la búsqueda del usuario.

```
#
# TABLA HASH – Directorio por Tipo
#
get '/directorio/:tipo' => sub ($c) {
    my $tipo = uc($c->param('tipo'));
    return $c->render(json => { ok=>0, mensaje=>'Tipo inválido (TIPO-01 a TIPO-04)' });
    unless $tipo =~ /^TIPO-0[1-4]$/;

    my $lista = $tablaHash->por_tipo($tipo);
    $c->render(json => { ok=>1, tipo=>$tipo, usuarios=>$lista, total=>scalar(@$lista) });
};

# Estado completo de la tabla hash (para reporte)
get '/directorio/estado' => sub ($c) {
    $c->render(json => { ok=>1, estado => $tablaHash->estado_tabla() });
};
```

Endpoints de chats LZW

Los endpoints que se encarga del óptimo funcionamiento del chat LZW, empiezan desde las siguientes líneas.

```
1550 #
1551 # MENSAJERÍA – Chats con LZW
1552 #
1553 #
1554 #
1555 # Enviar mensaje entre colaboradores
1556 post '/chat/mensaje' => sub ($c) {
1557     my $d = decode_json($c->req->body);
1558     my $de = trim($d->{de} // '');
1559     my $para = trim($d->{para} // '');
1560     my $msg = $d->{mensaje} // '';
1561
1562     return $c->render(json => { ok=>0, mensaje=>'Datos incompletos' });
1563     unless $de && $para && $msg;
1564
1565     unless ($grafo->existe_arista($de, $para)) {
1566         return $c->render(json => { ok=>0, mensaje=>'Solo puedes enviar mensajes a colaboradores directos' });
1567     }
1568 }
```

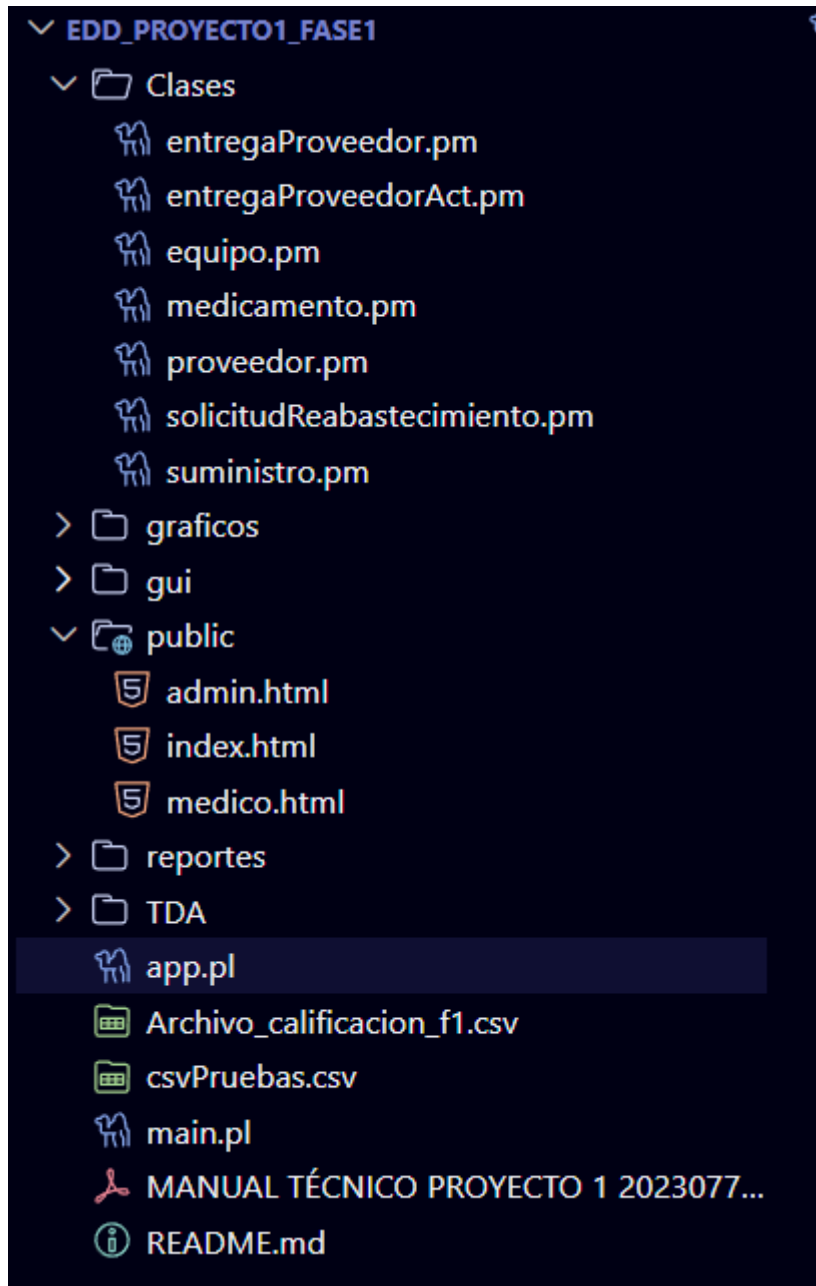
Endpoints de Solicitudes

Los endpoints que se encarga del óptimo funcionamiento de la parte de las solicitudes, empiezan desde las siguientes líneas.

```
1680 #
1681 # SOLICITUDES DE REABASTECIMIENTO (Lista Circular Doble)
1682 #
1683 #
1684 # POST: Crear nueva solicitud (Usuario)
1685 post '/solicitudes' => sub ($c) {
1686     my $d = decode_json($c->req->body);
1687     my $dep = trim($d->{departamento} // '');
1688     my $tipo = trim($d->{tipo_insumo} // '');
1689     my $cod = trim($d->{codigo} // '');
1690     my $cant = $d->{cantidad} // 0;
1691     my $mot = trim($d->{motivo} // '');
1692     my $sol = trim($d->{solicitante} // '');
1693
1694     # Validaciones estrictas
1695     return $c->render(json => { ok=>0, mensaje=>'Campos obligatorios: departamento, tipo, código y cantidad > 0' });
1696     unless $dep && $tipo && $cod && $cant > 0;
1697
1698     # Generar timestamp
```

Estructura de carpetas y consideraciones

Esta sería una imagen simple de cómo está estructurado este proyecto.



Consideraciones:

- Se debe instalar el framework de Mojolicious con ***Cpan mojolicious***.
- El código de JavaScript se trabajó dentro de una etiqueta de script en los archivos HTML5.